

PERSONAL AI AGENTS LIVE · JULY 21, 2026

Using Personal AI in 2026: OpenClaw and the New Developer Workflow

A 45-minute hands-on workshop: messy discovery to PRD, PRD to issues, issues to /goal, /goal to a working app.

Brad Groux

Digital Meld

OpenClaw Maintainer

YOUR FACILITATOR

Brad Groux

CEO of Digital Meld, OpenClaw contributor and maintainer, and builder of practical agent workflows that end in inspectable artifacts.



Builder, operator, maintainer

Digital Meld CEO, OpenClaw maintainer, and repo-backed demo builder.

 github.com/BradGroux

 twitter.com/BradGroux

 digitalmeld.io

THE PROBLEM

One-off prompts skip the work that makes software shippable.



Context gets lost

Stakeholder nuance, constraints, source confidence, and decision history disappear between chats.



Plans stay vague

Requirements, risks, acceptance criteria, and sequencing are implied instead of written down.



Code starts early

The agent builds before the team agrees what problem the product is supposed to solve.

THE SPLIT

OpenClaw plans. Codex builds. The repo preserves the trail.



OpenClaw

- Ingests stakeholder context
- Runs the grill-with-docs loop
- Forces missing decisions into the open
- Generates the PRD contract



Codex

- Turns PRD into issue artifacts
- Creates the /goal prompt
- Implements scoped work
- Runs verification and merges PRs

SCENARIO

E Corp Cyber Escalation Command Center

A Mr. Robot-inspired agentic delivery experience: discovery becomes a PRD, the PRD becomes issues and /goal prompts, and the work lands as a command center app.



First useful win

Angela filters critical escalations, checks reviewed evidence, assigns an owner, and exports a concise executive brief.

SOURCE CORPUS

The demo starts with a context packet, not code.

Source corpus means the working set of briefs, discovery notes, messages, systems, people, and signals OpenClaw reads before it grills the plan or drafts the PRD.



Briefs

Company shape, workshop goal, target users, operating constraints, and the first useful product outcome.



Signals

Stakeholder notes, escalation themes, evidence snippets, ownership gaps, and risk language the app must preserve.



Boundaries

What the demo may display, what stays draft-only, what requires human review, and what must never be automated.

\$GRILL-WITH-DOCS

The planning loop makes the hidden decisions explicit.



Users

Who triages, who decides, who reviews evidence, and who needs the brief?



Scope

What belongs in the first local app, and what stays out?



Safety

What must remain draft-only, fictional, reviewed, and local?



Proof

What would prove the workflow worked by the end of the workshop?

THE CONTRACT

The PRD becomes the handoff point.

It names the app, product output, presentation output, non-goals, app requirements, safety requirements, and acceptance criteria.

```
PRD.md
Problem
Goal
Product Output
Presentation Output
App Requirements
Safety Requirements
Acceptance Criteria
```


ISSUE ARTIFACTS

The PRD turns into scoped implementation work.



Escalation data

Risks, signals, severity, evidence, owners, statuses, and decision-ready executive summary fields.



Command workflow

Triage queue, evidence review, owner and status edits, executive brief, repo backlog, and Build Trace.



App proof

Local persistence, export JSON, reset path, responsive layout, smoke checks, and PR-backed delivery.

IMPLEMENTATION START

/goal is the durable start line.

It carries the PRD, issue artifacts, source corpus, app spec, ordering, safety boundary, and verification gates into the build loop. The source corpus is the context packet the agent can cite while building.

```
/goal Build the E Corp Cyber Escalation  
Command Center static app.
```

Read:

- PRD.md
- app-output-spec.md
- issue-artifacts.md
- source-corpus/ context packet

Verify:

- all tabs render
- filters and local state work
- export and reset work

THE ARTIFACT

A local command center, not a static report.

The app runs from local HTML, CSS, JavaScript, and JSON. It filters, edits local state, marks evidence reviewed, exports JSON, and resets cleanly.



PROOF

The Build Trace shows the artifact chain.

**Corpus**

Briefs, transcript, comms, signals.

**Grill**

Questions force scope and safety decisions.

**PRD**

Requirements become the contract.

**Issues**

Work becomes scoped and reviewable.

**/goal**

The build loop starts with context.

**App**

Local command center output.

**Export**

State is reviewable JSON.

This is the part attendees should copy: every jump in the chain has a checked-in artifact, a human review point, and a verification step. If the live run stalls, the Build Trace still shows exactly where to resume.

RECREATE IT

The repo documents the steps so attendees can run the pattern later.

→ Run the path

- Read the packet README
- Open the prompt pack
- Use the checked-in corpus if live generation is slow
- Start from the saved /goal prompt

🔔 Fallback cleanly

- Use the full corpus
- Use the implementation plan
- Use the finished app as proof
- Keep external action manual

CALL TO ACTION

Pick one messy workflow and turn it into a durable artifact chain.

Do not start with automation. Start with context, a PRD, scoped issues, a /goal prompt, and verification that proves the result.



Fork the packet

Replace the E Corp corpus with your own safe public or synthetic business context.



Run the grill

Use OpenClaw to force scope, safety, users, and proof into the PRD.



Ship one artifact

Use Codex to build the smallest inspectable app, report, or workflow that proves the method.

FOLLOW-UP

Start with a bounded workflow and leave a trail.

Use the repo packet, OpenClaw docs, and the SSTB.ai Discord to recreate the workflow after the event.

-  github.com/openclaw/openclaw
-  docs.openclaw.ai
-  github.com/BradGroux/openclaw-dev-days
-  www.bradgroux.com



Join the SSTB.ai Discord

discord.gg/Gmfkm7QVSF